SISTEMA BRT INTEGRADO

Documentação Técnica de Funções e Estruturas

Referência completa: o que cada parte do código faz e onde é usada

---

1. Visão Geral do Sistema

O Sistema BRT Integrado é um programa em linguagem C executado no terminal que gerencia operações de transporte público. Ele é dividido em três módulos de acesso — Administrador, Motorista e Controle Central — e persiste todos os dados em arquivos CSV compartilhados.

Arquivos de dados utilizados:

- `dados_onibus.csv` — Veículos cadastrados no sistema
- `dados_paradas.csv` — Paradas de ônibus cadastradas
- `dados_motoristas.csv` — Motoristas registrados
- `dados_linhas.csv` — Linhas e suas paradas associadas
- `avisos.csv` — Avisos enviados pelos motoristas
- `respostas.csv` — Respostas emitidas pelo controle central
- `sessoes.csv` — Jornadas abertas e encerradas pelos motoristas

---

2. Constantes e Macros (#define)

Definem os limites de arrays, separadores visuais e códigos de cor ANSI usados na interface do terminal.

Constante / Macro	Valor	Finalidade
`SEP`	`"====..."`	Linha de separação longa para cabeçalhos
`DIV`	`"----..."`	Linha divisória curta entre seções
`MAX`	`20`	Máximo de avisos por sessão de motorista
`MAX_AV`	`200`	Máximo de avisos carregados em memória (controle)
`MAX_RES`	`200`	Máximo de respostas carregadas em memória
`MAX_SES`	`100`	Máximo de sessões carregadas em memória
`COR_RESET`	`"\033[0m"`	Reseta cor do terminal para o padrão
`COR_VERM`	`"\033[31m"`	Texto vermelho (alertas críticos)
`COR_AMAR`	`"\033[33m"`	Texto amarelo (avisos de atenção)
`COR_VERD`	`"\033[32m"`	Texto verde (confirmações / OK)

COR_CIAN	"\033[36m"	Texto ciano (protocolos e destaques)
COR_BOLD	"\033[1m"	Texto em negrito no terminal

3. Estruturas de Dados (structs)

As structs modelam as entidades do sistema. Cada uma mapeia diretamente para uma linha de um arquivo CSV ou para um dado mantido em memória durante a execução.

3.1 Módulo do Motorista

Onibus

struct Onibus		
Campo	**Tipo**	**Descrição**
lin	int	Número da linha em que o ônibus opera
num	int	Número identificador do veículo
pla[10]	char[]	Placa do veículo
gar[50]	char[]	Nome da garagem responsável
mot[50]	char[]	Nome do motorista associado
tur[20]	char[]	Turno de operação (manha/tarde/noite)

Parada

struct Parada		
Campo	**Tipo**	**Descrição**
cod	int	Código único da parada
end[100]	char[]	Endereço completo da parada

Linha

struct Linha		
Campo	**Tipo**	**Descrição**
codLin	int	Código da linha de ônibus
codPar	int	Código de uma parada pertencente à linha

Motorista

struct Motorista		
Campo	**Tipo**	**Descrição**
nom[50]	char[]	Nome completo do motorista

cpf[15]	`char[]`	CPF do motorista (11 dígitos, apenas números)
tel[16]	`char[]`	Telefone de contato
lin	`int`	Número da linha de atuação

Aviso

`struct Aviso`		
Campo	**Tipo**	**Descrição**
prot[20]	`char[]`	Código de protocolo gerado automaticamente (ex: AV-0042-1234)
dth[25]	`char[]`	Data e hora do aviso no formato dd/mm/aaaa hh:mm:ss
tip[50]	`char[]`	Tipo do problema (Mecânico, Via, Passageiro, etc.)
grav[20]	`char[]`	Gravidade: Baixa, Média, Alta ou Urgente
loc[100]	`char[]`	Localização exata da ocorrência
desc[200]	`char[]`	Descrição detalhada do problema reportado

Sessao (sessão ativa do motorista)

`struct Sessao`		
Campo	**Tipo**	**Descrição**
mat[15]	`char[]`	Matrícula do motorista
nom[50]	`char[]`	Nome do motorista
lin[20]	`char[]`	Linha operada na jornada
vei[20]	`char[]`	Identificador do veículo utilizado
ini[25]	`char[]`	Data/hora de início da jornada
ativo	`int`	1 = jornada em andamento, 0 = encerrada
av[MAX]	`Aviso[]`	Array com até 20 avisos enviados na jornada atual
tot	`int`	Total de avisos enviados na sessão atual

3.2 Módulo do Controle Central

AvisoCC (aviso lido do CSV para o controle)

`struct AvisoCC`		
Campo	**Tipo**	**Descrição**
prot[20]	`char[]`	Protocolo do aviso
dth[25]	`char[]`	Data/hora do registro
mot[50]	`char[]`	Nome do motorista que enviou
lin[20]	`char[]`	Linha do motorista
tip[50]	`char[]`	Tipo do problema
grav[20]	`char[]`	Gravidade do problema

loc[100]	`char[]`	Local da ocorrência
desc[200]	`char[]`	Descrição do aviso
respondido	`int`	Flag: 1 se já tem resposta, 0 se ainda está pendente

Resposta

`struct Resposta`		
Campo	**Tipo**	**Descrição**
prot[20]	`char[]`	Protocolo do aviso ao qual esta resposta pertence
dth[25]	`char[]`	Data/hora em que a resposta foi emitida
oper[50]	`char[]`	Nome do operador que respondeu
acao[200]	`char[]`	Instrução ou ação determinada ao motorista

SessaoCC (sessão lida do CSV para o controle)

`struct SessaoCC`		
Campo	**Tipo**	**Descrição**
mat[15]	`char[]`	Matrícula do motorista
nom[50]	`char[]`	Nome do motorista
lin[20]	`char[]`	Linha operada
vei[20]	`char[]`	Veículo utilizado
ini[25]	`char[]`	Horário de início da jornada
fim[25]	`char[]`	Horário de encerramento (vazio se ainda ativo)
ativo	`int`	1 = jornada ativa, 0 = encerrada

Operador

`struct Operador`		
Campo	**Tipo**	**Descrição**
nom[50]	`char[]`	Nome do operador logado no controle central
mat[15]	`char[]`	Matrícula do operador

---

4. Funções Utilitárias (compartilhadas por todos os módulos)

Função	`limpar()`
Descrição	Limpa a tela do terminal para manter a interface organizada.
Parâmetros	Nenhum
Retorno	void

Usado em	Chamada no início de cabec() toda vez que uma nova tela é exibida

Função	`pausar()`
Descrição	Exibe a mensagem 'Pressione ENTER para continuar...' e aguarda o usuário antes de prosseguir.
Parâmetros	Nenhum
Retorno	void
Usado em	Ao final de quase todas as funções de interação, após exibir um resultado

Função	`dataHora(char *buf, int tam)`
Descrição	Obtém a data e hora atuais do sistema e grava no buffer no formato dd/mm/aaaa hh:mm:ss.
Parâmetros	buf: ponteiro para string destino; tam: tamanho do buffer
Retorno	void
Usado em	loginMotorista(), enviarAviso(), responderAviso(), relatorio(), menuControle()

Função	`trim(char *s)`
Descrição	Remove espaços e quebras de linha extras no início e no fim de uma string. Essencial para limpar entradas do usuário e linhas lidas de arquivos CSV.
Parâmetros	s: string a ser modificada in-place
Retorno	void
Usado em	Após leitura de strings de formulários e dentro das funções carregarAvisosCC(), carregarRespostas(), carregarSessoesCC()

Função	`cabec(const char *tit)`
Descrição	Limpa a tela e exibe o cabeçalho padrão do sistema com o título da tela atual.
Parâmetros	tit: string com o título da seção
Retorno	void
Usado em	No início de todas as funções de menu e formulário

Função	`ok(const char *m)`
Descrição	Exibe uma mensagem de sucesso prefixada com [OK].
Parâmetros	m: texto da mensagem
Retorno	void
Usado em	Após cadastros, envio de avisos e respostas com sucesso

Função	`erro(const char *m)`
Descrição	Exibe uma mensagem de erro prefixada com [!].

Parâmetros	m: texto da mensagem
Retorno	void
Usado em	Sempre que uma validação falha ou uma operação não pode ser concluída

Função	`aviso(const char *m)`
Descrição	Exibe uma mensagem de atenção prefixada com [~].
Parâmetros	m: texto da mensagem
Retorno	void
Usado em	Situações que exigem atenção mas não são erros (ex: jornada cancelada, avisos pendentes)

Função	`confirmar(const char *txt)`
Descrição	Exibe uma pergunta de confirmação (s/n) e retorna 1 se o usuário respondeu 'S' ou 's'.
Parâmetros	txt: texto da pergunta
Retorno	int: 1 (sim) ou 0 (não)
Usado em	cadOnibus(), cadParada(), cadMotorista(), cadLinha(), enviarAviso(), responderAviso()

Função	`vazio(const char *val, const char *campo)`
Descrição	Verifica se uma string está vazia e exibe mensagem de erro com o nome do campo caso esteja.
Parâmetros	val: conteúdo do campo; campo: nome do campo para exibir no erro
Retorno	int: 1 se vazio, 0 se válido
Usado em	cadOnibus(), cadParada(), cadMotorista(), enviarAviso(), responderAviso()

Função	`cpfOk(const char *c)`
Descrição	Valida se o CPF informado possui exatamente 11 caracteres, todos numéricos.
Parâmetros	c: string com o CPF
Retorno	int: 1 se válido, 0 se inválido
Usado em	cadMotorista()

Função	`placaOk(const char *p)`
Descrição	Valida se a placa tem pelo menos 7 caracteres.
Parâmetros	p: string com a placa
Retorno	int: 1 se válido, 0 se inválido
Usado em	cadOnibus()

Função	`corGrav(const char *g)`
Descrição	Retorna o código de cor ANSI correspondente à gravidade do aviso. Urgente/Alta = vermelho, Média = amarelo, Baixa = verde.
Parâmetros	g: string com a gravidade
Retorno	const char*: código de cor ANSI
Usado em	enviarAviso(), verRespostas(), historico(), exibirAvisoCC(), verPendentes(), responderAviso()

5. Funções de Carga de Dados em Memória

Estas funções leem os arquivos CSV e populam arrays de structs em memória. Utilizadas exclusivamente pelo módulo do Controle Central.

Função	`carregarAvisosCC(AvisoCC *lista, int max)`
Descrição	Lê o arquivo avisos.csv linha a linha, faz o parse de cada campo separado por ponto-e-vírgula e preenche o array lista. Inicializa o campo respondido como 0 em todos os itens.
Parâmetros	lista: array destino de AvisoCC; max: capacidade máxima do array
Retorno	int: total de avisos carregados
Usado em	verPendentes(), responderAviso(), historicoCompletoCC(), buscarProtocolo()

Função	`carregarRespostas(Resposta *lista, int max)`
Descrição	Lê o arquivo respostas.csv e preenche o array lista com protocolo, data, operador e ação de cada resposta registrada.
Parâmetros	lista: array destino de Resposta; max: capacidade máxima
Retorno	int: total de respostas carregadas
Usado em	verRespostas() (módulo motorista), verPendentes(), responderAviso(), historicoCompletoCC(), buscarProtocolo()

Função	`cruzarRespostas(AvisoCC *av, int totAv, Resposta *res, int totRes)`
Descrição	Faz o cruzamento entre avisos e respostas usando o protocolo como chave. Para cada aviso que tiver uma resposta correspondente, marca av[i].respondido = 1.
Parâmetros	av: array de avisos; totAv: total de avisos; res: array de respostas; totRes: total de respostas
Retorno	void
Usado em	verPendentes(), responderAviso(), historicoCompletoCC(), buscarProtocolo()

Função	`carregarSessoesCC(SessaoCC *lista, int max)`

Descrição	Lê o arquivo sessoes.csv e preenche o array com dados de cada jornada, incluindo status ativo/encerrado.
Parâmetros	lista: array destino de SessaoCC; max: capacidade máxima
Retorno	int: total de sessões carregadas
Usado em	verJornadas(), atualizarSessao()

Função	`atualizarSessao(const char *mat, const char *fim)`
Descrição	Lê todas as sessões, localiza a última jornada ativa da matrícula informada, registra o horário de encerramento e reescreve o arquivo sessoes.csv inteiro atualizado.
Parâmetros	mat: matrícula do motorista; fim: string com data/hora de encerramento
Retorno	void
Usado em	encerrar() (módulo motorista)

6. Módulo — Administrador

Acesso via usuário admin e senha 4321. Responsável pelo cadastro de toda a base de dados do sistema.

Função	`loginAdmin()`
Descrição	Solicita usuário e senha. Libera acesso ao painel administrativo apenas se as credenciais forem admin / 4321.
Parâmetros	Nenhum
Retorno	int: 1 se login aprovado, 0 se negado
Usado em	main() — caso 1

Função	`cadOnibus()`
Descrição	Formulário para cadastrar um novo ônibus com validação de linha (inteiro positivo), número, placa (mínimo 7 caracteres, com até 3 tentativas), garagem, motorista e turno. Após confirmação do usuário, grava no arquivo dados_onibus.csv.
Parâmetros	Nenhum
Retorno	void
Usado em	menuAdmin() — opção 1

Função	`cadParada()`
Descrição	Formulário para cadastrar uma parada com código numérico positivo e endereço. Após confirmação, grava em dados_paradas.csv.
Parâmetros	Nenhum
Retorno	void

Usado em	menuAdmin() — opção 2

Função	`cadMotorista()`
Descrição	Formulário para cadastrar motorista com nome, CPF (validado com cpfOk), telefone e linha. Grava em dados_motoristas.csv.
Parâmetros	Nenhum
Retorno	void
Usado em	menuAdmin() — opção 3

Função	`cadLinha()`
Descrição	Cadastra uma linha de ônibus associando-a a uma sequência de paradas, adicionadas uma a uma com opção de continuar. Grava em dados_linhas.csv no formato: codLinha;codParada1;codParada2;...
Parâmetros	Nenhum
Retorno	void
Usado em	menuAdmin() — opção 4

Função	`exibirArq(const char *arq, const char *tit)`
Descrição	Abre um arquivo CSV e imprime seu conteúdo bruto linha a linha no terminal, com um título formatado e contagem total de registros.
Parâmetros	arq: caminho do arquivo; tit: título da seção
Retorno	void
Usado em	relatorio()

Função	`relatorio()`
Descrição	Gera um relatório geral do sistema exibindo o conteúdo completo de todos os arquivos CSV: ônibus, paradas, motoristas, linhas, avisos e respostas.
Parâmetros	Nenhum
Retorno	void
Usado em	menuAdmin() — opção 5

Função	`menuAdmin()`
Descrição	Loop do painel do administrador. Exibe as opções de cadastro e relatório e direciona para a função correspondente até o usuário escolher voltar (opção 0).
Parâmetros	Nenhum
Retorno	void
Usado em	main() — após loginAdmin() bem-sucedido

7. Módulo — Motorista

Não exige senha — qualquer motorista pode fazer login informando seus dados. A sessão é registrada imediatamente no arquivo sessoes.csv.

Função	`loginMotorista()`
Descrição	Coleta matrícula, nome, linha e veículo do motorista. Registra o horário de início da jornada, inicializa a sessão global ses e grava uma linha em sessoes.csv com status 'ativo'.
Parâmetros	Nenhum
Retorno	void
Usado em	main() — caso 2

Função	`enviarAviso()`
Descrição	Permite ao motorista registrar um aviso ao controle. Verifica o limite de 20 avisos por sessão, gera um protocolo único no formato AV-<matrícula>-<número aleatório>, coleta tipo, gravidade, local e descrição, exibe resumo para confirmação e grava em avisos.csv.
Parâmetros	Nenhum
Retorno	void
Usado em	menuMotorista() — opção 1

Função	`verRespostas()`
Descrição	Percorre todos os avisos da sessão atual e busca respostas correspondentes em respostas.csv pelo protocolo. Exibe as respostas recebidas ou indica que ainda estão pendentes.
Parâmetros	Nenhum
Retorno	void
Usado em	menuMotorista() — opção 2

Função	`historico()`
Descrição	Exibe o histórico completo da jornada atual: motorista, hora de início e lista de todos os avisos enviados com protocolo, tipo, gravidade e local.
Parâmetros	Nenhum
Retorno	void
Usado em	menuMotorista() — opção 3

Função	`encerrar()`
Descrição	Exibe o resumo final da jornada, registra o horário de encerramento, chama atualizarSessao() para marcar a jornada como encerrada no CSV e desativa a sessão em memória.

Parâmetros	Nenhum
Retorno	void
Usado em	menuMotorista() — opção 0

Função	`menuMotorista()`
Descrição	Loop do painel do motorista. Exibe o nome, linha, veículo e contagem de avisos na barra de status e direciona para as funções até a jornada ser encerrada (opção 0).
Parâmetros	Nenhum
Retorno	void
Usado em	main() — após loginMotorista()

---

8. Módulo — Controle Central

Acesso restrito com até 3 tentativas. Possui dois usuários fixos: oper01 (Operador Central 01) e sup01 (Supervisor Central).

Função	`loginOperador()`
Descrição	Solicita matrícula e senha. Valida contra dois usuários fixos (oper01/brt2024 e sup01/sup2024). Preenche a struct global op_logado com o nome e matrícula do operador.
Parâmetros	Nenhum
Retorno	int: 1 se aprovado, 0 se negado
Usado em	main() — caso 3, com limite de 3 tentativas

Função	`exibirAvisoCC(const AvisoCC *a, int idx, int mostrarStatus)`
Descrição	Exibe os detalhes formatados de um aviso: protocolo, data/hora, motorista, linha, tipo, gravidade (com cor), local e descrição. Se mostrarStatus = 1, exibe também se está RESPONDIDO ou PENDENTE.
Parâmetros	a: ponteiro para o aviso; idx: número de ordem; mostrarStatus: flag para exibir status
Retorno	void
Usado em	verPendentes(), responderAviso(), historicoCompletoCC(), buscarProtocolo()

Função	`verPendentes()`
Descrição	Carrega avisos e respostas, faz o cruzamento e lista apenas os avisos que ainda não têm resposta, com contagem de pendentes x respondidos.
Parâmetros	Nenhum
Retorno	void

Usado em	menuControle() — opção 1

Função	`responderAviso()`
Descrição	Lista os avisos pendentes, permite o operador selecionar um, escolher uma instrução predefinida (opções A a F) ou digitar uma personalizada (opção G), confirmar e gravar a resposta em respostas.csv vinculada pelo protocolo.
Parâmetros	Nenhum
Retorno	void
Usado em	menuControle() — opção 2

Função	`verJornadas()`
Descrição	Carrega sessoes.csv e exibe separadamente as jornadas ativas (com matrícula, nome, linha, veículo e início) e as jornadas encerradas (com horário de fim).
Parâmetros	Nenhum
Retorno	void
Usado em	menuControle() — opção 3

Função	`historicoCompletoCC()`
Descrição	Exibe todos os avisos registrados no sistema com seus status e, quando existir, a resposta correspondente com nome do operador, data e instrução.
Parâmetros	Nenhum
Retorno	void
Usado em	menuControle() — opção 4

Função	`buscarProtocolo()`
Descrição	Solicita um código de protocolo ao operador e busca o aviso correspondente nos dados carregados. Exibe os detalhes completos e a resposta, se houver.
Parâmetros	Nenhum
Retorno	void
Usado em	menuControle() — opção 5

Função	`menuControle()`
Descrição	Loop do painel do controle central. Exibe o nome do operador e a hora atual em cada iteração, e direciona para as funções de gestão de avisos e jornadas.
Parâmetros	Nenhum
Retorno	void
Usado em	main() — após loginOperador() bem-sucedido

9. Função main()

Ponto de entrada do programa. Configura o locale para suporte a caracteres especiais e executa o loop principal do sistema.

Fluxo de execução:

- Exibe o menu de seleção de perfil: Administrador, Motorista, Controle Central ou Sair.
- Opção 1 (Admin): chama loginAdmin() e, se aprovado, abre menuAdmin().
- Opção 2 (Motorista): chama loginMotorista() sem verificação de senha e abre menuMotorista().
- Opção 3 (Controle): executa loginOperador() em loop com no máximo 3 tentativas. Se aprovado, abre menuControle(). Após 3 falhas, bloqueia o acesso.
- Opção 0: encerra o programa.
- Entradas inválidas: exibe mensagem de erro e retorna ao menu.